Tecnológico Nacional de México

Instituto Tecnológico de Chiná

Nombre de la carrera:

Ingeniería en Informática.

Nombre de la asignatura:

Programación cliente - servidor.

Semestre y grupo:

7AI

Actividad:

Transformar requisitos en recursos, operaciones y reglas de negocio antes de programar.

Nombre del alumno:

Aaron Jesús Eloy Aguilar León.

Chi Chablé Miguel Eduardo.

Ek Noh Edgar Ernesto.

Nombre del docente:

Jenner Noe Che Méndez.

Lugar y fecha:

San Francisco de Campeche, Campeche a 01 de Noviembre de 2026.

Objetivo o finalidad.

La finalidad por la cual se realizó esto, es debido a que se debe tener en cuenta la idea y de las funcionalidades de los RF para es el caso ahora del sistema de rancho que se seleccionó de manera previa en la unidad anterior, además del manejarlo así para llevar luego a cabo al uso de una API.

Diseño Previo de API: Prácticas Agropecuarias (Bitácora del Rancho Institucional).

1.- Descripción del problema.

En el rancho institucional las actividades agrícolas y pecuarias se realizan en zonas de campo donde existe conectividad intermitente a la red.

Los estudiantes y docentes necesitan registrar prácticas, observaciones, parcelas y cultivos de forma confiable sin perder información cuando falte señal de internet, requiriendo un sistema que permita almacenar los datos localmente y sincronizarlos con un servidor central de manera automática al recuperar la cobertura.

2.- Actores.

- Estudiante / Operador de Campo: Realiza y registra prácticas agropecuarias (riego, siembra, fertilización, cosecha) y observaciones técnicas en las parcelas asignadas.
- Docente / Administrador: Crea y administra las parcelas, gestiona los cultivos activos y supervisa los registros de la bitácora institucional.

3.- Requisitos funcionales.

El sistema debe permitir consultar el catálogo de parcelas agrícolas y cultivos vigentes.

El cliente móvil debe permitir el registro de prácticas agropecuarias y observaciones de campo aun sin conexión a la red.

El servidor debe proveer endpoints RESTful para procesar tanto registros individuales como la sincronización masiva de datos recolectados offline.

4.- Recursos identificados.

- parcelas: Terrenos o zonas de producción agrícola del rancho.
- cultivos: Especies vegetales o sembradíos activos asignados a las parcelas.
- practicas: Actividades agropecuarias y observaciones de campo realizadas por los usuarios.

5.- Atributos principales de cada recurso

Parcela: id, nombre, ubicacion, superficie_m2

Cultivo: id, parcela_id, nombre_cultivo, estado

Práctica: id, parcela_id, cultivo_id, tipo_practica, descripcion, fecha

6.- Relaciones entre recursos.

- Una Parcela puede tener uno o varios Cultivos históricos o activos (Relación 1 a N).
- Un Cultivo pertenece estrictamente a una Parcela específica.
- Una Práctica de campo está asociada obligatoriamente a una Parcela y a un Cultivo determinado.

7.- Operaciones.

- Consultar listado y detalle de parcelas y cultivos (GET).
- Registrar una nueva práctica de campo individual (POST).
- Enviar un lote masivo de prácticas acumuladas sin conexión (POST /sincronizar).

8.- Endpoints propuestos.

- GET /api/parcelas
- GET /api/cultivos
- POST /api/practicas
- POST /api/practicas/sincronizar

9.- Reglas de Negocio.

- Ninguna práctica agropecuaria puede registrarse si no está vinculada a un parcela_id y cultivo_id válidos en el sistema.
- Los datos sincronizados por lotes desde los dispositivos móviles deben validarse en el servidor para evitar la duplicidad de registros en la base de datos.
- Solo los usuarios con roles autorizados pueden emitir registros en las parcelas del rancho institucional.

10.- Validaciones.

El campo tipo_practica debe restringirse a valores permitidos (Siembra, Riego, Fertilización, Cosecha, Observación).

El campo fecha debe cumplir estrictamente con el formato de fecha y hora estándar (ISO 8601).

Los identificadores (parcela_id, cultivo_id) deben ser enteros positivos existentes en el sistema.

11.- Posibles errores.

- Enviar campos obligatorios vacíos o nulos en el cuerpo de la solicitud.
- Intentar registrar una práctica asociada a una parcela o cultivo inexistente.
- Conflictos en la sincronización masiva debido a registros duplicados o alterados.

12.- Códigos HTTP

- 200 OK: Petición o sincronización masiva procesada con éxito.
- 201 Created: Práctica agropecuaria registrada individualmente de forma exitosa.
- 400 Bad Request: Estructura JSON malformada o validación de campos obligatorios fallida.
- 404 Not Found: El recurso solicitado (parcela o cultivo) no existe.
- 409 Conflict: Conflicto de datos o intento de duplicidad en la sincronización.

13.- Ejemplo de una petición.

```JSON
{
  "parcela_id": 1,
  "cultivo_id": 3,
  "tipo_practica": "Riego",
  "descripcion": "Aplicación de riego por goteo en el sector norte.",
  "fecha": "2026-11-02T10:30:00Z"
}
```

14.- Ejemplo de una respuesta.

```JSON
{
  "estado": "EXITO",
  "codigo": 201,
  "mensaje": "Práctica agropecuaria registrada correctamente.",
  "data": {
    "id_practica": 105,
    "parcela_id": 1,
    "fecha_registro": "2026-11-02T10:30:00Z"
  }
}
```

Producto Final Esperado

El estudiante pasa de un requisito como:

"Un estudiante o docente puede registrar una práctica agropecuaria siempre que la parcela y el cultivo existan, y el dispositivo móvil sincronice la actividad al recuperar la conexión."

A algo estructurado:

RECURSOS

- Usuario (Estudiante / Docente)
- Parcela
- Cultivo
- Práctica (Bitácora de campo)

Después:

POST /api/practicas

Entrada:

```JSON
{
  "usuario_id": 12,
  "parcela_id": 3,
  "cultivo_id": 5,
  "tipo_practica": "Riego",
  "descripcion": "Aplicación de riego por goteo en la zona norte.",
  "fecha": "2026-11-02T10:30:00Z"
}
```

Reglas:

- RN-01: Usuario debe existir en el sistema institucional.
- RN-02: Usuario debe estar activo (estudiante o docente autorizado).
- RN-03: Parcela debe existir.
- RN-04: Cultivo debe existir y estar activo en dicha parcela.
- RN-05: Al registrar la práctica, la bitácora de campo almacena el evento (con soporte de sincronización offline).

Resultado:

201 Created.

Salida:

```json
{
  "id": 105,
  "usuario_id": 12,
  "parcela_id": 3,
  "cultivo_id": 5,
  "estado": "registrado"
}
```

Para el primer ejemplo de la sala de cómputo de la guía:

1.- Descripción del problema.

El sistema requiere gestionar de forma automatizada el control de préstamos de recursos y equipos a los usuarios, validando disponibilidad y límites de asignación mediante una API.

2.- Actores:

Administrador y Usuario / Estudiante.

3.- Requisitos Funcionales.

RF01: Permitir autenticación y gestión de usuarios.

RF02: Consultar disponibilidad de equipos.

RF03: Registrar solicitudes de préstamo cumpliendo las reglas de negocio.

4.- Recursos identificados.

/usuarios, /equipos, /prestamos.

5.- Atributos principales de cada recurso:

Usuario (id, nombre, estado), Equipo (id, nombre, estado), Préstamo (id, usuarioId, equipoId, fechaEsperadaDevolucion).

6.- Relaciones entre recursos:

Un usuario puede tener múltiples préstamos; un equipo puede ser asignado en distintos préstamos de forma secuencial.

7.- Operaciones.

GET
POST
PUT
DELETE.

8.-Endpoints propuestos:

POST /prestamos, GET /equipos, GET /usuarios.

9.- Reglas de negocio.

RN-01: Usuario debe existir.

RN-02: Usuario debe estar activo.

RN-03: Usuario debe tener menos de 2 préstamos.

RN-04: Equipo debe existir.

RN-05: Equipo debe estar disponible.

RN-06: Al crear préstamo, equipo pasa a "prestado".

10.- Validaciones.

Verificación de existencia de llaves foráneas y comprobación de límites activos antes de procesar el POST.

11.- Posibles errores.

400 Bad Request
401 Unauthorized
404 Not Found
500 Internal Server Error.

12.- Códigos HTTP.

200: OK.
201: Created.
400: Respuesta mala.
404: Recurso no encontrado.
500: Error interno del servidor.

13.- Ejemplo de una petición.

Método y Ruta: POST /prestamos

Cuerpo de Entrada (Request Body):

```
JSON

{
  "usuarioId": 25,
  "equipoId": 8,
  "fechaEsperadaDevolucion": "2026-10-08"
}
```[cite: 29]

14.- Ejemplos de una respuesta.

Código HTTP: 201 Created

Cuerpo de Respuesta (Response Body):

```
JSON

{
  "id": 152,
  "usuarioId": 25,
  "equipoId": 8,
  "estado": "activo"
}
```

Producto final esperado (Ejemplo original de la guía)

1.- Transición de requisito.

Requisito en lenguaje natural:

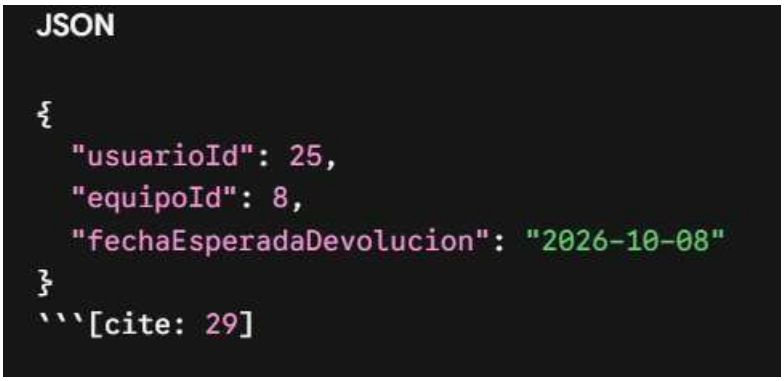
"Un estudiante puede solicitar un equipo siempre que tenga menos de dos préstamos y el equipo esté disponible."

Estructura formal de la API (Endpoint de Creación de Préstamo):

2.- Especificación de la Petición.

Método y Ruta: POST /prestamos

Cuerpo de Entrada (Request Body):



3.- Reglas de validaciones.

RN-01: Usuario debe existir.

RN-02: Usuario debe estar activo.

RN-03: Usuario debe tener menos de 2 préstamos activos.

RN-04: Equipo debe existir.

RN-05: Equipo debe estar disponible.

RN-06: Al crear préstamo, equipo pasa a "prestado".

4.- Resultados / Respuestas del servidor.

Código HTTP: 201 Created

Cuerpo de Respuesta (Response Body):



Para el caso práctico de ejemplo de citas de asesoría.

1.- Descripción del problema:

El sistema requiere gestionar de forma automatizada la reservación y control de citas de asesoría entre estudiantes y asesores, validando la disponibilidad de horarios y los límites de citas activas mediante una API REST.

2.- Actores:

Estudiante

Asesor

Administrador del sistema.

3.- Requisitos funcionales:

RF01: Permitir el registro y autenticación de usuarios.

RF02: Consultar la disponibilidad de horarios de los asesores.

RF03: Registrar nuevas citas de asesoría cumpliendo las reglas de negocio establecidas.

4.- Recursos identificados:

/usuarios

/asesores /citas.

5.- Atributos principales de cada recurso:

Usuario/Estudiante (id, nombre, estado), Asesor (id, nombre, especialidad, estado), Cita (id, estudianteId, asesorId, fechaHoraAsesoria, estado).

6.- Relaciones entre recursos:

Un estudiante puede agendar múltiples citas; un asesor puede recibir múltiples citas en diferentes horarios disponibles.

7.- Operaciones:

GET,

POST

PUT

DELETE.

8.- Endpoints propuestos:POST /citas, GET /asesores/disponibilidad, GET /citas/estudiante/{id}.

9.- Reglas de negocio:

RN-01: El estudiante debe existir.

RN-02: El estudiante debe estar activo.

RN-03: El estudiante debe tener menos de 2 citas de asesoría activas.

RN-04: El asesor debe existir.

RN-05: El asesor debe estar disponible en el horario seleccionado.

RN-06: Al crear la cita, el horario del asesor pasa a estado "reservado".

10.- Validaciones:

Verificación de la existencia de llaves foráneas (estudianteId, asesorId) y comprobación de que el horario no esté previamente ocupado antes de procesar el POST.

11.- Posibles errores:

400 Bad Request

401 Unauthorized

404 Not Found

500 Internal Server Error.

12.- Códigos HTTP:

200: OK.

201: Created.

400: Respuesta mala.

404: Recurso no encontrado.

500: Error interno del servidor.

13.- Ejemplo de una petición:

Método y Ruta: POST /citas

Cuerpo de Entrada (Request Body):

```JSON
{
  "estudianteId": 25,
  "asesorId": 4,
  "fechaHoraAsesoria": "2026-11-05T10:00:00Z"
}
```

14.- Ejemplos de una respuesta:

Código HTTP: 201 Created

Cuerpo de Respuesta (Response Body):

```JSON
{
  "id": 152,
  "estudianteId": 25,
  "asesorId": 4,
  "estado": "activa"
}
```

Producto final esperado: Transformación de Requisitos a API (Citas de Asesoría).

1.- Transición del requisito.

Requisito en lenguaje natural:

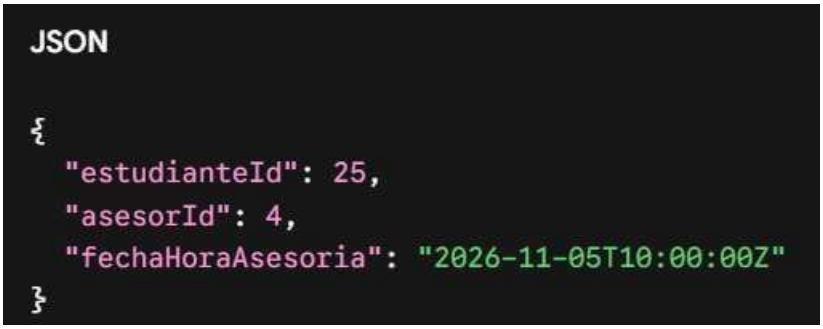
"Un estudiante puede agendar una cita de asesoría siempre que tenga menos de dos citas activas y el asesor esté disponible en el horario seleccionado."

Estructura formal de la API (Endpoint de Creación de Cita):

2.- Especificación de la Petición

Método y Ruta: POST /citas

Cuerpo de Entrada (Request Body):



3.- Reglas de Negocio Validadas

RN-01: El estudiante debe existir.

RN-02: El estudiante debe estar activo.

RN-03: El estudiante debe tener menos de 2 citas activas.

RN-04: El asesor debe existir.

RN-05: El asesor debe estar disponible.

RN-06: Al crear la cita, el horario del asesor pasa a "reservado".

4.- Resultado / Respuesta del Servidor

Código HTTP: 201 Created

Cuerpo de Respuesta (Response Body):

```JSON
{
  "id": 152,
  "estudianteId": 25,
  "asesorId": 4,
  "estado": "activa"
}
```